Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

PickleVerse – Multiplayer Pickleball Skill-Training and Match Game with AI Coaching

Project code: PKV Group: GFA26SE111

Supervisor: Mr. Lại Đức Hùng

Team: Huỳnh Trung Kiên (Leader), Nguyễn Văn Thành Đạt,

Nguyễn Gia Huy, Trần Nguyễn Hiếu Thuận

– Ho Chi Minh City, November 2026 –

Table of Contents

I. Record of Changes 3

II. Project Management Plan 3

1. Overview 3

1.1 Scope & Estimation 3

1.2 Project Objectives 5

1.3 Project Risks 6

2. Management Approach 7

2.1 Project Process 7

2.2 Quality Management 9

2.3 Training Plan 10

3. Project Deliverables 10

4. Responsibility Assignments 11

5. Project Communications 12

6. Configuration Management 12

6.1 Document Management 12

6.2 Source Code Management 12

6.3 Tools & Infrastructures 13

I. Record of Changes

Date	A* M, D	In charge	Change Description
27/10/2026	A	Nguyễn Gia Huy	v0.1 – Work packages, schedule and risks summarised for the Review 2 design brief (BM01, BM04).
06/11/2026	A	Nguyễn Văn Thành Đạt	v0.2 – First complete Report 2 on the official template for Review 3.
07/11/2026	M	Nguyễn Văn Thành Đạt	v0.3 – Added weekly effort, package readiness and review checks.
11/11/2026	M	Nguyễn Gia Huy	v1.0 – Revised after the internal review: two-level WBS with complexity and effort; quality targets in the template format; risks with owner and trigger; actual iterations; dated milestones with actual dates; RACI; training plan; configuration management that matches the repository.

*A - Added M - Modified D - Deleted

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

The scope of version 1.0 is the ten major features FE-01 to FE-10 defined in Report 1, plus the technical work that the features depend on: the gameplay core, the networking layer, art and audio assets, deployment and testing. Table 1 decomposes this scope into a two-level work breakdown structure (WBS). Each function is classified as Simple, Medium or Complex and estimated in man-days, where one man-day is eight hours of work by one member.

Table 1. Work breakdown structure and effort estimation

#	WBS Item	Complexity	Est. Effort (man-days)	Owner
1	**Project management and documentation**		**28**	
1.1	Planning, scheduling and weekly tracking	Medium	5	Kiên
1.2	Report 1 and Report 2	Simple	3	Huy, Kiên
1.3	Report 3 – Software Requirement Specification	Complex	8	Huy
1.4	Report 4 – Software Design Document	Complex	7	Đạt
1.5	Reports 5–7, user guides and defense slides	Medium	5	Thuận, Kiên
2	**Foundation and architecture**		**20**	

#	WBS Item	Complexity	Est. Effort (man-days)	Owner
2.1	Unity project, scenes and assembly structure	Simple	3	Huy
2.2	Layered Web API, EF Core model and database schema (51 tables)	Complex	9	Đạt
2.3	Docker Compose environment with PostgreSQL, Redis and S3 storage	Medium	4	Đạt
2.4	CI/CD pipeline (GitHub Actions, Docker Hub, SSH deployment)	Medium	4	Đạt
3	**Gameplay core**		**30**	
3.1	Ball flight and bounce simulation at 120 Hz	Complex	8	Huy
3.2	Paddle contact, shot input and seven shot kinds	Complex	7	Huy
3.3	Singles rules engine (scoring, serve, faults, non-volley zone)	Complex	7	Huy
3.4	Character movement and animation controller	Medium	5	Huy
3.5	Match flow, HUD, intro cinematic and result screen	Simple	3	Thuận
4	**FE-01 Account and onboarding**		**16**	
4.1	Registration and OTP email verification	Medium	4	Đạt
4.2	Sign in, sign out, JWT access and refresh tokens	Medium	4	Đạt
4.3	Password recovery	Simple	2	Thuận
4.4	Profile, character and onboarding level	Simple	2	Thuận
4.5	Settings: audio, graphics, key binding, touch layout	Medium	4	Thuận
5	**FE-02 Tutorial and technique training**		**23**	
5.1	Interactive tutorial (17 steps)	Medium	5	Huy
5.2	Lesson and drill framework with star scoring	Complex	7	Huy
5.3	Content of 15 lessons and 12 techniques	Medium	5	Huy
5.4	Offline training and one-time synchronisation	Complex	6	Đạt
6	**FE-03 Bot practice**		**15**	
6.1	Fixed-level bot (Easy, Medium, Hard)	Medium	5	Huy
6.2	Adaptive behaviour-tree bot with Elo tiers	Complex	7	Đạt
6.3	Bot match setup, court and lighting options, rewards	Simple	3	Thuận
7	**FE-04 Online matches**		**36**	
7.1	Photon Fusion session, lobby and room codes	Complex	7	Đạt
7.2	Ranked matchmaking, ready check and bot fallback	Complex	7	Đạt
7.3	Dedicated server build, signed tickets, lease and result	Complex	9	Đạt
7.4	State synchronisation, input prediction and reconnection	Complex	9	Đạt
7.5	Elo rating and match result persistence	Medium	4	Đạt
8	**FE-05 AI coaching**		**18**	
8.1	Telemetry capture, local spool and upload	Medium	5	Đạt
8.2	FastAPI rule engine and gated tier model	Complex	7	Đạt
8.3	Hybrid analysis worker, analysis jobs and coaching panel	Medium	6	Huy
9	**FE-06 Progress, rewards and shop**		**15**	
9.1	XP, levels, technique mastery and progress dashboard	Medium	5	Huy
9.2	Daily and weekly missions, achievements, streak rewards	Medium	5	Đạt
9.3	Currencies, shop and collection	Medium	5	Huy
10	**FE-07 Social, history and ranking**		**12**	
10.1	Friends, blocking and room invitations	Medium	4	Đạt
10.2	Match history, replay recording and replay viewer	Complex	6	Đạt
10.3	Elo leaderboard with cache	Simple	2	Kiên
11	**FE-08 Reports, restrictions and appeals**		**6**	
11.1	Player report after a match	Simple	2	Kiên
11.2	Restricted access, decision view and appeal	Medium	4	Kiên
12	**FE-09 Administration web application**		**24**	
12.1	Admin authentication, layout and protected routing	Simple	2	Kiên
12.2	User management, sanctions and progress reset	Complex	6	Kiên
12.3	Content management: lessons, references, cosmetics, missions	Medium	6	Kiên
12.4	Live match monitoring and stopping	Medium	4	Đạt
12.5	Moderation of player reports and appeals	Medium	3	Kiên
12.6	Analytics dashboard	Medium	3	Kiên
13	**FE-10 Expert review**		**4**	
13.1	Shot labelling and dataset export	Medium	4	Đạt
14	**Art, audio and interface assets**		**15**	

#	WBS Item	Complexity	Est. Effort (man-days)	Owner
14.1	Characters, 20 animation clips, courts and stadium	Complex	8	Huy
14.2	Music and sound effects	Simple	2	Kiên
14.3	Front-end screens, loading artwork and Vietnamese fonts	Medium	5	Thuận
15	**Testing and release**		**18**	
15.1	Unit tests (Unity EditMode, Python) and coverage	Medium	5	Thuận
15.2	Integration and system testing on staging	Medium	5	Thuận
15.3	Acceptance tests, playtests and defect fixing	Medium	6	All
15.4	Staging deployment and release package	Simple	2	Đạt
	Total Estimated Effort (man-days)		**280**	

The estimate covers 56 functions (11 Simple, 28 Medium and 17 Complex) and totals **280 man-days**. The available capacity is four members × 14 weeks × 5 working days = 280 man-days, so the plan uses the whole capacity and has no reserve. The team accepts this because Complex networking items were estimated from the actual effort of the October prototypes, and AI coding assistants are used for boilerplate code and first drafts of documents; every generated output is reviewed by the member who owns the item. If an iteration overruns, the first items to be deferred are FE-10 (expert review) and quick doubles matching, which are not part of the core workflows.

1.2 Project Objectives

The overall objective is to deliver, by Week 14 (13/12/2026), a deployed and tested version of PickleVerse in which a beginner can learn the rules, train the core techniques with graded drills and feedback, and play bot and online matches, while administrators manage accounts, content, matches and moderation from the web application. The quality, time and cost objectives are measurable as follows.

Quality. Table 2 states the planned test coverage and the expected number of defects at each testing stage. The defect estimate (about 120 defects in total) is based on the defects already found in the October prototypes and the 10/11/2026 playtest; it is used to judge whether a stage is finding fewer defects than expected, which usually means that testing is too shallow.

Table 2. Quality objectives by testing stage

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
1	Reviewing	100% of reports, diagrams and database changes are reviewed by at least one other member	20	17%	Document and design review before each submission
2	Unit Test	100% of public methods in the rule, scoring, Elo, input and economy classes; ≥70% line coverage of core gameplay assemblies	30	25%	Unity EditMode and Python pytest; run before every merge
3	Integration Test	All 122 API endpoints exercised at least once; all database write paths	25	21%	IntegrationChecks harness and API contract checks in CI
4	System Test	100% of the 117 test cases in Report 5 executed on staging	35	29%	Includes network, device and AI quality tests
5	Acceptance Test	8 demo scenarios covering workflows W01–W04	10	8%	Executed with the supervisor before the final review
	Total		**120**	**100%**	

In addition, the release criteria for version 1.0 are: at least 90% of system test cases passed, no open Critical or High defect, at least 50% of test cases executed automatically, and every functional requirement covered by at least one passed test case.

Milestone timeliness. Up to 11/11/2026, six milestones (M01–M06) were due and one of them (M06) was delivered on time, a timeliness of 17%. Reports 1–4 were first submitted on 06/11/2026 because the team prioritised the playable prototype in Weeks 5–8 (§2.1). The target for the remaining milestones M07–M10 is 100% on time, and the corrective actions are: one document owner per report, a weekly tracking meeting with the leader, and submission one day before each deadline.

Cost (allocated effort). The 280 man-days are distributed over the project activities as shown in Table 3. Feature and technical items include their own design (15%), coding (70%) and unit testing (15%); documentation items are counted under requirements, design or project management.

Table 3. Allocated effort by activity

Activity	Source in the WBS	Effort (man-days)	Share
Requirements	1.2, 1.3	11	3.9%
Design	1.4 + 15% of items 2–14	42	15.0%
Coding	70% of items 2–14	164	58.6%
Testing	15% of items 2–14 + item 15	53	18.9%
Project management and documentation	1.1, 1.5	10	3.6%
Total		**280**	**100%**

1.3 Project Risks

Risks are reviewed in the weekly tracking meeting. A risk that occurs is closed in this table and recorded as an issue in the Issues sheet of the project tracking workbook.

Table 4. Project risks

#	Risk Description	Impact	Possibility	Trigger	Response Plans	Owner
1	Key-person dependency: most of the networking, backend and AI code has been written by one member, so the online features stall if that member is unavailable.	High	Medium	Owner unavailable for more than 3 days, or one member has more than 60% of the commits in an iteration	Each member owns and demonstrates one workflow; pair-programming sessions on Fusion and the API; handover notes in Doc/; code walkthrough at the end of each iteration	Kiên
2	Leaked credentials: the SMTP password and JWT signing key were committed in appsettings.json and could be misused.	High	High	Secret found in a tracked file (occurred; see issue I-03)	Move secrets to environment variables and the server .env file; rotate the leaked values; add a secret scan to CI	Đạt
3	One dedicated game-server slot limits concurrent online matches and the scalability target.	High	High	"Server busy" responses during tests with more than two players	Bot fallback and a clear busy message; load-test the API separately; document a server pool as future work	Đạt
4	The Photon free plan limits concurrent users, which can block group playtests or the demo.	Medium	Medium	Connection refused when more than 20 clients connect	Use the dedicated server path for matches; schedule playtests; upgrade the plan before the defense if needed	Đạt
5	Unity tests cannot run in GitHub Actions without a Unity licence, so automation evidence is weak.	Medium	High	CI shows no Unity test step	Run EditMode tests with the Unity CLI and attach logs with the commit; evaluate a GameCI runner with a personal licence	Thuận
6	Resetting the development database deletes test accounts and data.	Medium	Medium	Schema change that requires a reset	Keep seed accounts and content in Database.sql; back up with pg_dump before a reset; re-run seed checks	Đạt
7	Documents or code produced with AI assistants contain statements that do not match the implementation.	High	Medium	A reviewer finds a claim without evidence in code or tests	The owner of each section verifies it against code and test results; cross-document consistency check before submission	Huy

#	Risk Description	Impact	Possibility	Trigger	Response Plans	Owner
8	Too little real player data to validate the AI tier model.	Medium	High	Fewer than 200 labelled sessions by Week 12	Keep the tier model gated; use rule-based findings; collect labels with the expert review tool during playtests	Đạt
9	High latency or packet loss between Vietnamese players and the single staging host degrades matches.	Medium	Medium	Round-trip time above 100 ms or rubber-banding reported in playtests	Measure RTT in each playtest; tune interpolation; reconnection within 30 seconds	Đạt
10	Schedule slip and scope creep (milestones M01–M05 were late).	High	High	A milestone is not ready two days before its date	Freeze scope after Review 3; weekly tracking; defer FE-10 and quick doubles first	Kiên
11	The staging host fails and data is lost (single server, no replica).	High	Low	Host unreachable or disk failure	Daily database backup; redeploy from Docker Hub images with deploy.sh; keep a local Docker environment as fallback	Đạt
12	The game does not reach the frame-rate target on mid-range Android phones.	Medium	Medium	Average below 30 FPS on the test phone	Profile with the Unity Profiler; quality presets; reduce stadium detail and shadows on mobile	Huy

2. Management Approach

2.1 Project Process

The team follows an iterative and incremental process with two-week iterations, adapted from Scrum for a student team. The leader acts as Scrum Master and product owner on behalf of the supervisor; each member owns a technical area and one end-to-end workflow. Every iteration starts with planning from the product backlog in the tracking workbook and ends with a demonstration to the team and, at review weeks, to the supervisor. Figure 1 shows the cycle.

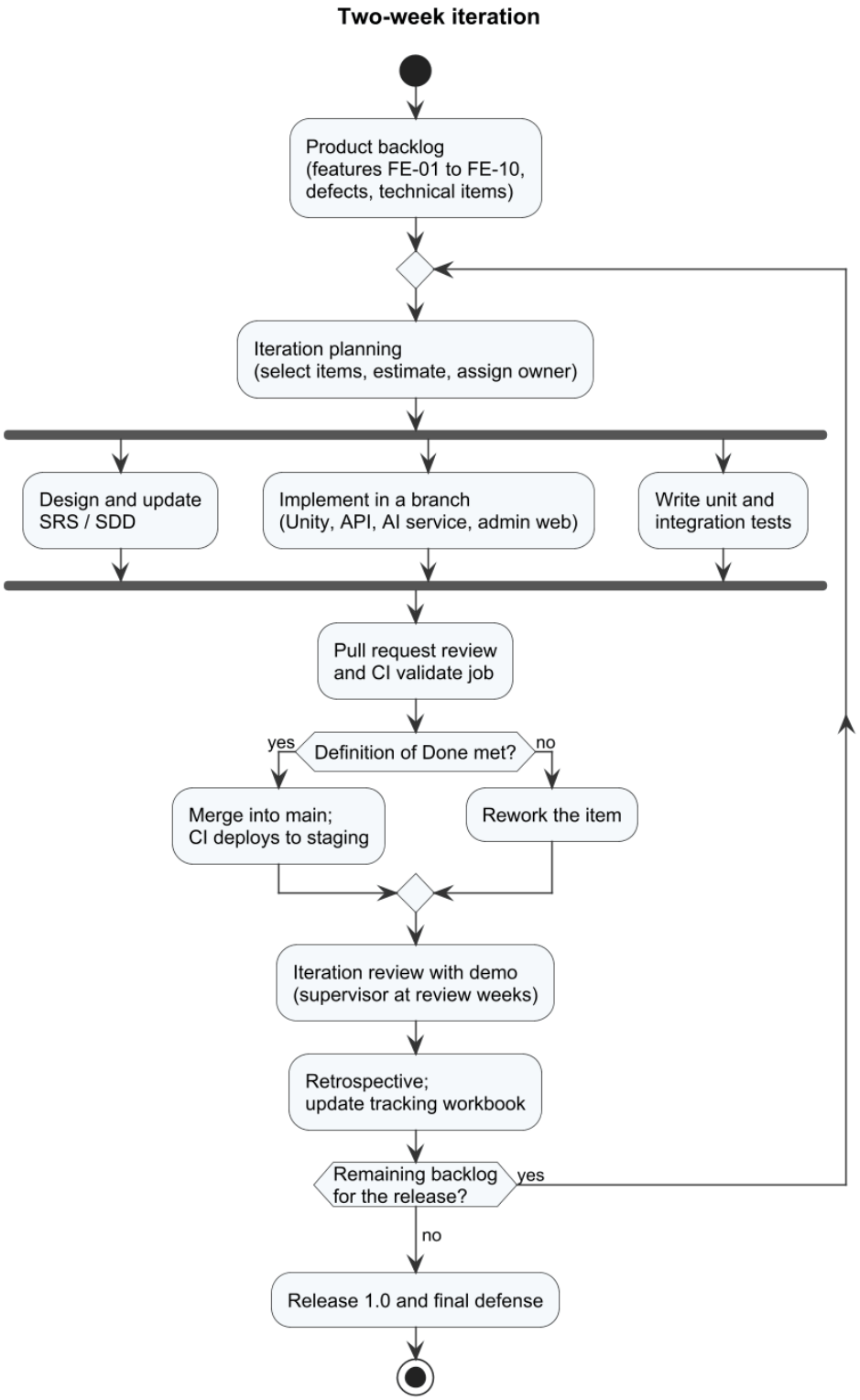


Figure 1. Iterative development process used by the team

Definition of Done. A backlog item is done when (1) the code is merged into the main branch, (2) its unit tests pass and the existing EditMode and Python suites still pass, (3) the related API, database and UI changes are consistent with Database.sql and the API client, (4) it works on the staging server, and (5) the SRS, SDD and test cases are updated.

The actual iterations, reconstructed from the repository history, are listed in Table 5. The architecture changed three times, which explains why the early milestones were late: the online layer moved from a Netcode for GameObjects prototype to Photon Fusion in shared mode and then to an authoritative dedicated server, and player data moved from local JSON files to the Web API and PostgreSQL.

Table 5. Iterations and actual progress

Iteration	Weeks and dates	Main results	Status
It-0 Inception	W1–W2, 07/09–20/09	Proposal and team set-up; repository created (first commit 23/08); Unity project initialised (17/09)	Done
It-1 Prototype	W3–W4, 21/09–04/10	Unity project structure and camera; 3D character models generated and rigged (03/10)	Done
It-2 Single-player core	W5–W6, 05/10–18/10	20 animation clips; BotMatch singles with rules (07/10); mode selection; music and SFX (09/10); shop and collection (10/10); tutorial (11/10); training mode (12/10); history, missions, Elo and ranks (11/10); friends and leaderboard (12/10); replay and streaks (13/10); Netcode for GameObjects prototype (07/10–21/10)	Done
It-3 Online and backend	W7–W8, 19/10–01/11	Loading scene and touch controls; Web API and database (20/10); move to Photon Fusion (20/10); Unity–API integration (21–23/10); admin web (22–24/10); OTP email (24/10); doubles and custom rooms (25/10); Review 2 package (27/10); AI service and Unity Behavior bot (27/10); normalised schema and ApiResponse refactor (31/10); login scene (01/11)	Done
It-4 Dedicated server and coaching	W9–W10, 02/11–15/11	Dedicated server, accounts and coaching (03/11); Docker deployment (04/11); CI/CD to Docker Hub and staging (05–06/11); admin tools and response training (06/11); reports v0.2 (06/11); reconnect and offline fixes (10/11); playtest (10/11); 515 EditMode tests passed (11/11); reports v1.0 (11/11)	In progress
It-5 Stabilisation	W11–W12, 16/11–29/11	Fix the 18 open defects of test Round 1 (Report 5); secrets out of the repository; pull-request workflow; system and load tests; Report 5 results; Review 3	Planned
It-6 Release	W13–W14, 30/11–13/12	Acceptance tests and demo scenarios; release package; Report 6 user guides; Report 7; defense rehearsal (W15)	Planned

2.2 Quality Management

Defect prevention: Coding rules are written in AGENTS.md and enforced in review: the layered backend architecture (API → Services → Repositories → Entities), access to data only through the unit of work, ApiResponse<T> envelopes, a single schema source (src/Database.sql) and no secrets in source files. Gameplay rules are implemented in engine-independent C# classes so that they can be unit-tested.

Reviewing: Every report section is written by its owner and reviewed by one other member against the code and test results before submission (Table 8). Until 10/11/2026 code was committed directly to the main branch without pull requests; from Iteration 5 every change is merged through a pull request that needs one reviewer and a passing CI run (§6.2).

Unit testing: Unity EditMode tests (515 tests in 10 assemblies, all passed on 11/11/2026) cover the rules, scoring, Elo, input, economy, replay and multiplayer protocol classes; pytest covers the AI service. Tests run before each merge and the results are stored with the commit hash.

Integration testing: The IntegrationChecks console program and the API contract checks run in the CI validate job against a PostgreSQL and Redis instance; they verify the API, the database writes and the dedicated-server ticket flow.

System testing: The 117 test cases in Report 5 are executed on the staging server for each package milestone; defects are logged in the tracking workbook with severity and retested before closing.

Acceptance testing: The eight demo scenarios of the Review 2 commitment are run with the supervisor, and the results are recorded in Report 5.

2.3 Training Plan

Table 6. Training plan

Training Area	Participants	When, Duration	Waiver Criteria
Git and GitHub flow (branches, pull requests, reviews)	All members	W1, 2 hours; refresher W11, 1 hour	Mandatory
Unity 6, C# gameplay scripting, Input System	Thuận, Kiên	W2–W3, 12 hours	Has completed a Unity game project before
Photon Fusion 2 and dedicated server builds	Huy, Thuận	W8–W9, 8 hours (taught by Đạt)	Has run and debugged a networked match on the staging server
ASP.NET Core Web API, EF Core and PostgreSQL	Kiên, Huy	W6–W7, 8 hours (taught by Đạt)	Has implemented an API endpoint with a repository and an EF Core query
React 18, TypeScript and React Query	Kiên, Thuận	W7, 6 hours	Has built a React application with TypeScript
Python FastAPI and scikit-learn	Đạt, Huy	W8, 4 hours	Has deployed a FastAPI service
Docker, Docker Compose and GitHub Actions	All members	W9, 4 hours	Can start the local stack and read container logs
Blender and character animation import	Huy, Thuận	W4, 6 hours	Has rigged and imported a character into Unity
UML modelling with PlantUML and draw.io	All members	W3, 3 hours	Mandatory

3. Project Deliverables

Week 1 starts on Monday 07/09/2026; each milestone is due on the Sunday of its week. Actual dates are taken from the repository history and submission records.

Table 7. Project deliverables and milestones

#	Deliverable	Due Date	Actual Date	Status	Notes
M01	Report 1 – Project Introduction	13/09/2026	06/11/2026	Done, late	Proposal registered before Week 1; v1.0 on 11/11/2026
M02	Report 2 – Project Management Plan, schedule, tracking workbook	20/09/2026	06/11/2026	Done, late	v1.0 on 11/11/2026
M03	Report 3 – Software Requirement Specification	27/09/2026	06/11/2026	Done, late	v1.0 on 11/11/2026
M04	Report 4 – Software Design Document; Report 5 test plan	11/10/2026	06/11/2026	Done, late	v1.0 on 11/11/2026
M05	Software Package 1 – single-player core, training, first online build (Review 2)	25/10/2026	27/10/2026	Done, 2 days late	Review 2 forms BM01–BM04
M06	Software Package 2 – Web API, admin web, AI service, dedicated server, CI/CD	08/11/2026	06/11/2026	Done, on time	Staging deployment on tad-min.io.vn
M07	Software Package 3 – defect fixes, system and load tests (Review 3)	22/11/2026	–	In progress	Report 5 results due with this package
M08	Full software package with test results and open defects	06/12/2026	–	Planned	
M09	Report 6 – Software User Guides	13/12/2026	–	Planned	Installation, player and administrator guides
M10	Report 7 – Final Project Report, release package and defense	20/12/2026	–	Planned	Week 15

Figure 2 places the iterations and milestones on the calendar; the red line marks the date of this version (11/11/2026).

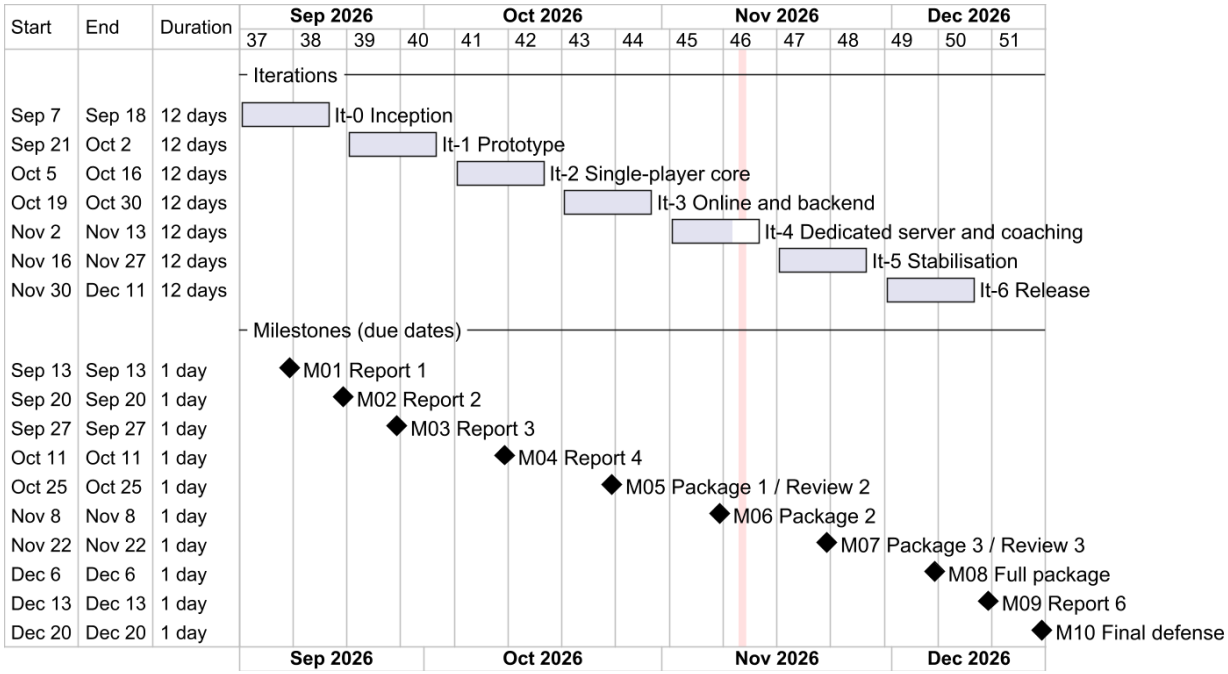


Figure 2. Master schedule: iterations and milestones

4. Responsibility Assignments

D~Do; R~Review; S~Support; I~Informed; <blank>~Omitted. Every row has exactly one D. Each member owns one end-to-end workflow, which is demonstrated by that member at the reviews.

Table 8. Responsibility assignment matrix (RACI)

Responsibility	**KienHT SE184454**	**DatNVT SE184450**	**HuyNG SE184439**	**ThuanTNH SE184471**
Project planning and tracking	D	R	S	S
Report 1 – Project Introduction	R	S	D	I
Report 2 – Project Management Plan	D	S	R	I
Report 3 – SRS: overview, use cases, screens	R	S	D	S
Report 3 – SRS: functional and non-functional requirements	S	R	D	S
Report 4 – SDD: architecture, database, API, security	I	D	R	S
Report 4 – SDD: gameplay, bot and client design	I	R	D	S
Report 5 – Test plan, test cases and test report	R	S	S	D
Report 6 – User guides	D	S	R	S
Report 7 – Final report and defense slides	D	R	R	S
Gameplay core, training and tutorial	I	R	D	S
Online matches, dedicated server and netcode	I	D	R	S
Web API, database and AI coaching service	S	D	R	I
Admin web application	D	R	I	S
Unity user interface, settings and loading	S	I	R	D
3D art, animation and courts	I	I	D	S
Music and sound effects	D	I	R	I
CI/CD and staging deployment	I	D	S	R
Workflow W01 – Learn and sync a drill	R	S	D	I
Workflow W02 – Play a bot and get a tip	I	S	R	D
Workflow W03 – Find and play an online match	R	D	S	I
Workflow W04 – Report, sanction and appeal	D	S	I	R

5. Project Communications

Table 9. Project communications

Communication Item	Who/ Target	Purpose	When, Frequency	Type, Tool, Method(s)
Supervisor meeting	Team and Mr. Lại Đức Hùng	Demonstrate progress, agree on scope changes, record decisions and answers	Weekly, 60 minutes; extra meeting before each review	Online or on-campus meeting; minutes and decisions in the Q&A sheet
Iteration planning	All members	Select backlog items, estimate and assign owners	First day of each two-week iteration	Meeting; Iteration backlog in the tracking workbook
Progress check	All members	Report done work, next work and blockers	Twice a week (Tuesday and Friday evening)	Group chat and short call
Iteration review and retrospective	All members, supervisor at review weeks	Demonstrate the increment; decide improvements	Last day of each iteration	Meeting; notes in the tracking workbook
Code review	Author and one reviewer	Check correctness, tests and architecture rules	Each pull request (from Iteration 5)	GitHub pull request with CI result
Defect and issue updates	Owner of the defect or issue	Keep status, severity and owner current	When status changes; reviewed weekly	Tracking workbook (Defects and Issues sheets)
Report submission	Leader	Submit reports and packages	One day before each deadline	FPT University LMS

6. Configuration Management

6.1 Document Management

- All project documents are stored in the Git repository under `Doc/`: the proposal in `Doc/Proposal`, review forms in `Doc/Project_report/Review*`, and Reports 1–7 with the tracking workbook and schedule in `Doc/Project_report/Report`.
- File names follow the course template: `Report<n>_<Title>.docx`, `PickleVerse_Project_Tracking.xlsx` and `PickleVerse_Project_Schedule.xml`.
- Each document records its versions in the Record of Changes table: v0.x for drafts and v1.0, v1.1 and so on for reviewed versions submitted to the supervisor.
- Diagrams are written as PlantUML sources in `Doc/Project_report/Report/diagrams/src` and rendered to PNG, so every figure can be regenerated and reviewed as text; the use case and conceptual ERD drawings are also kept as draw.io files.
- A document is submitted only after it has been reviewed by the member marked R in Table 8 and the leader; the PDF exported from the reviewed version is uploaded to the LMS.

6.2 Source Code Management

- Repository: GitHub `Tad-Min/PickleVerse`, one repository for the Unity client (`My project/`), the backend and admin web (`src/`), deployment files (`deploy/`, Docker Compose files) and documents.
- Practice until 10/11/2026: members committed directly to `main` and merged local work with merge commits; the coaching work used the branch `realtime-AI-response`. No pull requests or release tags were used, so reviews were informal.
- Practice from Iteration 5 (16/11/2026): every change is made on a branch named `feature/<item-id>-<short-name>` or `fix/<defect-id>`, merged into `main` through a pull request with one reviewer and a passing CI validate job. Review packages are tagged `review-3` and `v1.0`; the Review 2 package is tagged retrospectively at the commit of 27/10/2026.

- Commit messages follow the Conventional Commits style (`feat:`, `fix:`, `chore:`, `docs:`) and reference the backlog item or defect ID.
- Large binary sources (Blender work files, local Unity work folders) are excluded by `.gitignore`; only exported models, textures and animation clips are committed.
- Secrets are not stored in source files: connection strings, the JWT signing key, the SMTP password, Photon keys and the dedicated-server service key are supplied through environment variables and the server `.env` file.
- Each push to `main` runs the GitHub Actions workflow `dockerhub.yml`, which validates the backend, AI service and web build, publishes images to Docker Hub (`tadminm/pickelverse`) and deploys them to the staging host by image digest.

6.3 Tools & Infrastructures

Table 10. Tools and infrastructure

Category	Tools / Infrastructure
Technology	Unity 6000.6.2f1 with C# (game client and dedicated server); Photon Fusion 2.1.3; Unity Behavior 1.0.16; ASP.NET Core on .NET 10 (Web API); Python 3.12 with FastAPI and scikit-learn (AI service); React 18, TypeScript 5, Vite 5 and React Query (admin web)
Database	PostgreSQL 16 (Npgsql Entity Framework Core); Redis 7.4 (matchmaking queue, leases, presence, cache); S3-compatible object storage (replays)
IDEs/Editors	Unity Editor and Unity CLI, Visual Studio Code, Visual Studio, pgAdmin 4, Docker Desktop, Photon Dashboard
Art and audio	Blender (rigging and animation), Meshy AI (base character meshes), Figma (interface mock-ups)
Diagramming	PlantUML with Graphviz, draw.io
Documentation	Microsoft Office (Word, Excel, Project-compatible XML), Markdown
Version Control	Git and GitHub (source code and documents)
CI/CD and deployment	GitHub Actions, Docker and Docker Compose, Docker Hub; staging host tad-min.io.vn (Ubuntu 24.04, nginx)
Testing	Unity Test Framework (EditMode), Unity CLI, pytest, IntegrationChecks console harness, Swagger UI; k6 is planned for the API load test (NFR-04)
Project management	Tracking workbook (backlog, WBS, issues, defects, Q&A) and the MS Project XML schedule; GitHub for code and pull requests
AI assistants	OpenAI Codex and Anthropic Claude for code suggestions and first drafts of documents; every output is reviewed by the item owner